

Clase 2: Configuración del Entorno y Fundamentos Documentales

Modalidad Remota: 100% Hands-On

Parte 1

Verificación del Entorno y Troubleshooting

Verificación de Instalación Local

1. El Servidor (mongod)

Confirme que el motor de base de datos está corriendo en segundo plano.

- » Verifique la existencia del directorio de datos (ej. C:\data\db).
- » Si lo ejecuta manualmente, busque el mensaje: **"waiting for connections"**.
- » Puerto por defecto: **27017**.

2. El Cliente (mongosh)

Abra una **nueva terminal** y lance el intérprete moderno de JavaScript.

```
> mongosh
> show dbs
admin      40.00 KiB
config     60.00 KiB
local      40.00 KiB
```

Nota: El comando antiguo mongo está obsoleto.

"Power Tools" de la Terminal



Ayuda Integrada (Help)

Utilice `db.help()` para ver los métodos a nivel de base de datos y `db.collection.help()` para ver todo lo que puede hacer con una colección en particular.



Introspección de Funciones

Si escribe el nombre de una función **sin paréntesis** (ej. `db.users.insertOne`), la terminal imprimirá el código fuente JavaScript de la función. Muy útil para verificar el orden de los parámetros.

Parte 2

Conceptos Centrales y BSON

El "Cambio de Chip": Jerarquía



Base de Datos

Es el contenedor físico para las colecciones.
Cada base de datos tiene su propio conjunto de archivos en el disco.

(Equivale a Database)



Colección

Grupo de documentos.
No impone un esquema rígido; los documentos adentro pueden ser totalmente distintos.

(Equivale a Tabla)



Documento

La unidad básica de datos (pares clave-valor). Requiere obligatoriamente un campo `_id` único.

(Equivale a Fila/Registro)

Anatomía BSON (Binary JSON)

¿Por qué BSON?

JSON es texto plano, excelente para humanos pero lento para máquinas. BSON es binario, aportando:

- » **Velocidad de Parsing:** Salto rápido entre campos.
- » **Tipos de datos ricos:** Soporte para Fechas nativas, Enteros de 32/64 bits, Decimales y ObjectIds.
- » **Límite de Tamaño:** 16 Megabytes por documento máximo.

Sensibilidad Estricta

MongoDB es sensible a tipos y a mayúsculas/minúsculas.

```
// Estos 3 documentos son considerados
// ESTRUCTURALMENTE DIFERENTES por MongoDB:

{ "edad": 25 }
{ "Edad": 25 }
{ "edad": "25" }
```

El Misterio del ObjectId



Generación Distribuida

El campo `_id` actúa como **Primary Key**. Si no lo proporciona, MongoDB generará un `ObjectId` automáticamente.

Mide 12 bytes y está diseñado para ser rápido de generar y garantizar unicidad a nivel global en sistemas distribuidos.



Timestamp Integrado

Los primeros 4 bytes de un `ObjectId` representan un *Timestamp UNIX*.

Consecuencia genial: Al ordenar por `_id`, los documentos ya están naturalmente ordenados por su fecha de creación exacta.

Parte 3

Operaciones CRUD Modernas

Creación y Lectura (C y R)

Create: insertOne() y insertMany()

```
db.users.insertOne({
  "name": "Alice",
  "rol": "Backend"
})
```

La API moderna divide la inserción explícitamente entre uno y varios documentos para evitar errores.

Read: find() y findOne()

```
// Devuelve un cursor (Lote de docs)
db.users.find({ "rol": "Backend" })

// Devuelve solo el primer documento
db.users.findOne({ "name": "Alice" })
```

En el shell mongosh moderno, los resultados ya vienen formateados prolijamente por defecto.

Actualización y Borrado (U y D)

Update: La importancia del \$set

```
// Modificación Atómica (Correcto)
db.users.updateOne(
  { "name": "Alice" },
  { $set: { "age": 30 } }
)
```

Si olvida el \$set, MongoDB arrojará error. Para **reemplazar todo el documento** use explícitamente `replaceOne()`.

Delete: deleteOne() y deleteMany()

```
// Borra el primero que encuentre
db.users.deleteOne({ "name":
"Alice" })

// Limpia TODA la colección
db.users.deleteMany({})
```

Operaciones seguras y explícitas que reemplazan al obsoleto comando `remove()`.

Parte 4

Laboratorios Prácticos (Hands-on)

Elija su Laboratorio



Opción A: Identidad Digital

Foco: Esquemas dinámicos.
Manejo de documentos asimétricos, sub-documentos anidados, arrays y operadores condicionales (`$gt`, `$in`).



Opción B: E-Commerce

Foco: Tipos BSON estrictos.
Asegurar la integridad de datos en stock con `Int32`, `Long`, `Date`.
Modificaciones atómicas matemáticas (`$inc`).



Opción C: Motor de Blog

Foco: Datos Embebidos.
Búsqueda profunda en arrays (Notación de Puntos) y manipulación manual de estado como ODM.



Desafío Integrador Extra



El Carrito de Compras

Diseñe la estructura de un documento que represente el "Carrito de Compras" activo de un usuario y realice las operaciones para gestionarlo.



Requisitos

Deberá ser capaz de inicializar el carrito en `$0`, **añadir** productos al array usando `$push`, actualizar el precio total simultáneamente con `$set`, y **remover** items si el usuario se arrepiente usando `$pull`.

Cheat Sheet: Comandos Fundamentales

Base de Datos & Colecciones

```
show dbs //
Lista las bases de datos
use <db_name> //
Cambia o crea base de datos
db.dropDatabase() //
Elimina base de datos actual

show collections //
Lista las colecciones
db.<coll>.drop() //
Elimina la colección
```

Operaciones CRUD

```
// CREATE & READ
db.coll.insertOne({ doc })
db.coll.find({ filtro })

// UPDATE & DELETE
db.coll.updateOne({ filtro },
{ $set: { ... } })
db.coll.deleteOne({ filtro })
```

Cheat Sheet: Operadores Clave

Comparación

\$eq / \$ne : Igual / Distinto

\$gt / \$gte : > / >=

\$lt / \$lte : < / <=

\$in / \$nin : En/No en arreglo

```
{ edad: { $gte: 18 } }
```

Lógicos

\$and : Cumple todas

\$or : Cumple al menos una

\$exists : El campo existe

\$all : Contiene todos

```
{ $or: [{ stock: 0 }, { activo: false } ] }
```

Actualización

\$set : Modifica campo

\$unset : Elimina campo

\$inc : Incrementa

\$push / \$pull : Add/Quit Array

```
{ $push: { hobbies: "leer" } }
```



Más Desafíos Prácticos



D2: Sistema de Reservas

Modele un documento para un **Vuelo**. Debe contener: destino, límite de asientos y un array de pasajeros.

- » Cree un vuelo vacío hacia "París" con límite de 2 lugares.
- » Añada a 2 pasajeros diferentes al array usando `$push`.
- » Actualice el estado del vuelo a "Completo" condicionalmente buscando por su `_id`.



D3: IoT & Sensores

Simule la ingesta de datos de un sensor. Inserte un lote de 5 lecturas de temperatura usando `insertMany()`.

- » Lecturas de ejemplo: 22.5, 25.1, 30.2, 18.0, 31.5.
- » Escriba un `find()` para reportar solo las temperaturas mayores a 30 grados (**Alerta**).
- » Utilice `deleteMany()` para purgar todas las lecturas menores a 20 grados.

Cierre y Validación



Salida Segura (Shutdown)

NUNCA cierre la ventana del servidor mongod haciendo clic en la "X" abruptamente.

Vaya a la terminal del servidor y presione **Ctrl + C** para permitir que el motor escriba los últimos datos en disco y libere el archivo `mongod.lock`.



Tarea: Preparación Clase 3

La próxima clase (Presencial) subiremos el nivel hacia arquitecturas distribuidas y Alta Disponibilidad.

Lectura obligatoria: Leer bibliografía sobre la Conjetura de Brewer (Teorema CAP) y prepararse para debatir por qué MongoDB prioriza consistencia y tolerancia a particiones (CP).